

Architectural Map — EMS Readiness Optimization

Generated: 2026-03-21

Repository: `ems-optimization/`

Package version: 0.6.0 (`src/ems_readiness/__init__.py`)

Methodology: Repository-to-architecture reconstruction. All claims are evidence-based and classified as **Confirmed**, **Inferred**, or **Unknown**.

A. Executive Summary

- **Purpose:** This repository implements a simulation-based approach to optimizing ambulance (EMS) staging locations across Manhattan's 48 FDNY firehouses, aiming to minimize emergency response times. **[Confirmed — `README.md` , `docs/core/technical_report.md`]**
- **Core technique:** Three allocation policies (P0 spatially-stratified baseline, P1 demand-proportional, P2 MIP-optimized) are evaluated via discrete-event simulation (SimPy) with NHPP demand modeling calibrated from 2.24M historical motor vehicle collision records. **[Confirmed — `scripts/run_production_v2.py` , `src/ems_readiness/`]**
- **Key result:** The optimized policy P2 achieves a 19% reduction in mean response time (3.17 → 2.57 min) over P0 at K=20 with capacity=2. **[Confirmed — `README.md` , `results/baseline/tables/descriptive_statistics.csv`]**
- **Canonical pipeline entry point:** `python scripts/run_production_v2.py` — runs allocation generation, 810 simulation runs (3 policies × 9 K values × 30 replications), statistical analysis, and visualization. **[Confirmed — source code inspection]**
- **Data pipeline entry point:** `python scripts/generate_all_data.py` (or `make data`) — processes raw NYC Open Data into demand lambda tables and distance matrices. **[Confirmed — `Makefile` , `source code`]**
- **Project type:** Batch pipeline (data processing → optimization → simulation → statistical analysis → visualization). Not a service, not notebook-driven. **[Confirmed — no server code, no `setup.py` entry points]**
- **Reproducibility:** Deterministic seeding via `SeedManager` (master seed 42), 30 replications per scenario, all configs in YAML. **[Confirmed — `configs/reproducibility.yaml` , `src/ems_readiness/utils/reproducibility.py`]**
- **Technology stack:** Python 3.11+, PuLP/CBC (MIP solver), SimPy (DES), pandas/numpy/scipy, geopandas, matplotlib/seaborn. **[Confirmed — `requirements.txt`]**
- **Git LFS:** Two large raw CSVs (Motor Vehicle Collisions ~2.2M rows, Police Precincts) are tracked via Git LFS. **[Confirmed — `.gitattributes`]**
- **Results organization:** `results/baseline/` is the canonical source of truth; `results/analysis/` contains supporting robustness/sensitivity analyses; `results/archive/` is legacy (cap=5 era). **[Confirmed — `results/WHICH_FILES_TO_USE.md`]**

B. Repository Execution Map

What to Run First

Prerequisite: Ensure processed data exists. Most processed data files are committed (demand lambdas, distance matrices, firehouses CSVs, precincts GeoJSON). The crash CSV and parquet are `.gitignore`d but are only needed for data regeneration, not for the production pipeline.

Step	Command	Purpose	Prerequisites	Evidence
0 (optional)	<code>python -m venv venv && source venv/bin/activate && pip install -r requirements.txt</code>	Create environment	Python 3.11+	<code>Makefile</code> target <code>setup</code>
1 (if needed)	<code>python scripts/generate_all_data.py</code> or <code>make data</code>	Generate processed data from raw	Raw CSV files in <code>data/raw/</code>	<code>scripts/generate_all_data.py</code>
1a (verify)	<code>python scripts/generate_all_data.py --verify</code> or <code>make verify-data</code>	Verify processed data exists	None	<code>Makefile</code>
2 (primary)	<code>python scripts/run_production_v2.py</code>	Run full production pipeline	Processed data in <code>data/processed/</code>	<code>scripts/run_production_v2.py</code>
3 (optional)	<code>python scripts/run_verification.py</code>	Run V&V toy scenarios	Processed data	<code>scripts/run_verification.py</code>
4 (optional)	<code>python scripts/run_validation_pilots.py</code>	Run validation pilots	Processed data	<code>scripts/run_validation_pilots.py</code>
5 (optional)	<code>pytest tests/ -v</code> or <code>make test</code>	Run unit/integration tests	Processed data, full dependencies	<code>Makefile</code> , <code>tests/</code>

Required Inputs Before First Execution

Input	Path	Status	Notes
Firehouse listing	data/raw/FDNY_Firehouse_Listing_20260223.csv	Committed	Small file, always present
Manhattan boundary	data/raw/manhattan_boundary.geojson	Committed	Geometry file
CBD boundary	data/raw/cbd_boundary.geojson	Committed	For CBD analyses
NYC borough boundaries	data/raw/nyc_borough_boundaries.geojson	Committed	Borough filtering
Crash data	data/raw/Motor_Vehicle_Collisions_-_Crashes_20260223.csv	Git LFS (~2.2M rows)	Needed for generate_all_data.py
Police precincts	data/raw/Police_Precincts_20260223.csv	Git LFS	Needed for generate_all_data.py
Demand lambda tables	data/processed/demand_lambda_*.csv	Committed (gitignore exception)	Pre-computed; usable directly
Distance matrices	data/processed/distance_matrix_*.csv	Committed (gitignore exception)	Pre-computed; usable directly
Firehouses (clean)	data/processed/firehouses_*.csv	Committed (gitignore exception)	Pre-computed
Precincts GeoJSON	data/processed/precincts_manhattan.geojson	Committed (gitignore exception)	Pre-computed

Key insight: The production pipeline (`run_production_v2.py`) can run without the raw LFS data because the essential processed files (demand lambdas, distance matrices, firehouses) are committed to the repo as gitignore exceptions. **[Confirmed — `.gitignore` exception rules]**

C. Environment and Dependency Requirements

Runtime Version

- **Python:** 3.11+ [**Confirmed** — `README.md` badge, `__pycache__` `dirs show` `cpython-311`]

Dependency File

- `requirements.txt` at project root [**Confirmed**]

Required Libraries (from requirements.txt)

Library	Version	Role	Evidence
pandas	≥2.0.0	Data manipulation	Confirmed — requirements.txt, all scripts
numpy	≥1.24.0	Numerical computing	Confirmed
geopandas	≥0.14.0	Geospatial data processing	Confirmed — data pipeline
matplotlib	≥3.7.0	Visualization	Confirmed — run_production_v2.py
seaborn	≥0.12.0	Statistical visualization	Confirmed — requirements.txt
scipy	≥1.11.0	ANOVA, Tukey HSD, distributions	Confirmed — run_production_v2.py
simpy	≥4.0.0	Discrete-event simulation	Confirmed — simulation/engine.py
pulp	≥2.7.0	MIP solver (CBC backend)	Confirmed — optimization/models.py
shapely	≥2.0.0	Geometric operations	Confirmed — geopandas dependency
pyproj	≥3.6.0	Coordinate transforms	Confirmed — geopandas dependency
folium	≥0.15.0	Interactive maps	Confirmed — requirements.txt
tqdm	≥4.65.0	Progress bars	Confirmed — requirements.txt
joblib	≥1.3.0	Parallel processing (data pipeline)	Confirmed — requirements.txt
openpyxl	≥3.1.0	Excel data dictionary reading	Confirmed — requirements.txt
jupyter / jupyter-lab	≥1.0/4.0	Notebook execution	Confirmed — requirements.txt

Configuration Files

Config	Path	Loaded By	Purpose
Demand	configs/demand.yaml	ar-rival_generator.py	Base arrival rate, lambda table paths
Optimization	configs/optimization.yaml	allocator.py	K values, capacity, solver settings
Service	configs/service.yaml	travel_time.py , service_time.py	Speed, time-of-day factors, service distribution
Simulation	configs/simulation.yaml	engine.py , runner.py	Horizon, replications, thresholds
CBD Scenario	configs/cbd_scenario.yaml	run_cbd_experiment.py	CBD precinct lists, experiment scenarios
Reproducibility	configs/reproducibility.yaml	reproducibility.py	Master seed, deterministic mode

Config loading mechanism: YAML files loaded via `yaml.safe_load()` with- in respective modules. The `EMSAllocator.from_project()` factory reads configs relative to a `project_root` path. `run_production_v2.py` hardcodes key constants (`CAPACITY=2`, `K_VALUES`, etc.) and passes them to library calls, partially overriding config file values. **[Confirmed — source code inspection]**

Potential contradiction: `configs/optimization.yaml` sets `default_K: 40` and `firehouse_capacity: 2`, while `run_production_v2.py` hardcodes `K_VALUES = [10,15,20,25,30,35,40,45,48]` and `CAPACITY = 2`. The script's hardcoded values take precedence at runtime. **[Confirmed — code > config]**

Environment Variables

- None explicitly referenced in source code. **[Confirmed — grep found no `os.environ` or `os.getenv` calls in src/ or scripts/]**

External Services / System Tools

- **CBC solver:** Bundled with PuLP (no external install needed). **[Confirmed — `configs/optimization.yaml` specifies `CBC`]**
- **Git LFS:** Required only for pulling raw crash/precinct CSVs. Not needed if processed data is already present. **[Confirmed — `.gitattributes`]**

D. View 1: Business Context Diagram

```

graph TB
    subgraph External_Data_Sources
        NYC_OD["NYC Open Data<br/>(Motor Vehicle Collisions,<br/>FDNY Firehouses,<br/>Police Precincts)"]
        GEO["NYC Boundary GeoJSONs<br/>(Manhattan, CBD, Boroughs)"]
    end

    subgraph "EMS Readiness Optimization System"
        DP["Data Processing Pipeline<br/>scripts/generate_all_data.py"]
        DM["Demand Model (NHPP)<br/>src/ems_readiness/demand/"]
        OPT["MIP Optimizer (PuLP/CBC)<br/>src/ems_readiness/optimization/"]
        SIM["DES Engine (SimPy)<br/>src/ems_readiness/simulation/"]
        STAT["Statistical Analysis<br/>(ANOVA, Tukey HSD, CI)"]
        VIZ["Visualization Engine<br/>(matplotlib/seaborn)"]
    end

    subgraph Stakeholders
        PM["EMS Operations Planners"]
        RES["Academic Researchers"]
        ENG["Engineers / Maintainers"]
    end

    subgraph Outputs
        ALLOC["Allocation Tables<br/>results/baseline/allocations/"]
        METRICS["Performance Metrics<br/>results/baseline/tables/"]
        FIGS["Publication Figures<br/>results/baseline/figures/"]
        REPORT["Technical Report<br/>docs/core/technical_report.md"]
    end

    NYC_OD --> DP
    GEO --> DP
    DP --> DM
    DM --> OPT
    DM --> SIM
    OPT --> SIM
    SIM --> STAT
    STAT --> VIZ
    OPT --> ALLOC
    STAT --> METRICS
    VIZ --> FIGS
    METRICS --> REPORT
    FIGS --> REPORT

    ALLOC --> PM
    METRICS --> PM
    FIGS --> RES
    REPORT --> RES
    ALLOC --> ENG

    style NYC_OD fill:#e3f2fd
    style GEO fill:#e3f2fd
    style ALLOC fill:#c8e6c9
    style METRICS fill:#c8e6c9
    style FIGS fill:#c8e6c9
    style REPORT fill:#c8e6c9

```

E. View 2: Data Lineage / I/O Diagram

```

graph LR
  subgraph "Raw Inputs (data/raw/)"
    R1["FDNY_Firehouse_Listing_20260223.csv<br/>(committed)"]
    R2["Motor_Vehicle_Collisions_-_Crashes_20260223.csv<br/>(Git LFS, 2.24M rows)"]
    R3["Police_Precincts_20260223.csv<br/>(Git LFS)"]
    R4["manhattan_boundary.geojson<br/>(committed)"]
    R5["cbd_boundary.geojson<br/>(committed)"]
  end

  subgraph "Tier 1: Boundaries (scripts/data_processing/tier1_boundaries.py)"
    T1["Process boundaries<br/>→ geometry pickles"]
  end

  subgraph "Tier 2: Spatial Filtering"
    T2a["tier2_firehouses.py<br/>→ firehouses_manhattan.csv"]
    T2b["tier2_precincts.py<br/>→ precincts_manhattan.geojson"]
    T2c["tier2_crashes.py<br/>→ crashes_manhattan.csv/.parquet"]
  end

  subgraph "Tier 3: Modeling"
    T3a["tier3_demand.py<br/>→ demand_lambda_hourly.csv<br/>→ demand_lambda_dow.csv<br/>→ demand_lambda_precinct.csv<br/>→ demand_model_summary.json"]
    T3b["tier3_distance.py<br/>→ distance_matrix_firehouse_precinct.csv<br/>→ distance_matrix_firehouse_precinct_manhattan.csv"]
  end

  subgraph "Processed Data (data/processed/) – committed exceptions"
    P1["firehouses_manhattan.csv"]
    P2["firehouses_clean.csv"]
    P3["precincts_manhattan.geojson"]
    P4["demand_lambda_hourly.csv"]
    P5["demand_lambda_dow.csv"]
    P6["demand_lambda_precinct.csv"]
    P7["distance_matrix_firehouse_precinct.csv"]
    P8["demand_model_summary.json"]
  end

  subgraph "Optimization (src/ems_readiness/optimization/)"
    OPT_A["allocator.py<br/>+ models.py + policies.py"]
  end

  subgraph "Simulation (src/ems_readiness/simulation/)"
    SIM_E["engine.py → dispatcher.py<br/>→ metrics.py → entities.py"]
  end

  subgraph "Baseline Outputs (results/baseline/)"
    O1["allocations/allocations_K{10..48}.csv"]
    O2["simulation/all_results_raw.csv<br/>+ results_K{10..48}.csv"]
    O3["tables/descriptive_statistics.csv<br/>+ anova_results.csv<br/>+ pos-thoc_comparisons.csv<br/>+ effect_sizes.csv<br/>+ confidence_intervals.csv<br/>+ queue_statistics.csv"]
    O4["figures/mean_rt_vs_K.png<br/>+ coverage_vs_K.png<br/>+ allocation_map_K*.png<br/>+ pub_fig*.png"]
  end

  R4 --> T1
  R5 --> T1
  T1 --> T2a
  T1 --> T2b
  T1 --> T2c
  R1 --> T2a

```

```

R3 --> T2b
R2 --> T2c
T2a --> P1
T2a --> P2
T2b --> P3
T2c --> T3a
T3a --> P4
T3a --> P5
T3a --> P6
T3a --> P8
P1 --> T3b
P3 --> T3b
T3b --> P7

P4 --> SIM_E
P5 --> SIM_E
P6 --> SIM_E
P7 --> OPT_A
P6 --> OPT_A
P1 --> OPT_A

OPT_A --> 01
01 --> SIM_E
SIM_E --> 02
02 -->|"statistical analysis<br/>(run_production_v2.py Step 3)"| 03
02 -->|"visualization<br/>(run_production_v2.py Step 4)"| 04

style R2 fill:#fff3e0
style R3 fill:#fff3e0
style P4 fill:#e8f5e9
style P5 fill:#e8f5e9
style P6 fill:#e8f5e9
style P7 fill:#e8f5e9
style 01 fill:#c8e6c9
style 02 fill:#c8e6c9
style 03 fill:#c8e6c9
style 04 fill:#c8e6c9

```

Legend: Orange = Git LFS tracked (external dependency); Green = committed / output artifacts.

F. View 3: Runtime Sequence Diagram

Golden Path: `python scripts/run_production_v2.py`

```
sequenceDiagram
    participant User
    participant RunProdV2 as scripts/run_production_v2.py
    participant Allocator as EMSAllocator<br/>(optimization/allocator.py)
    participant Policies as policies.py
    participant Models as models.py (PuLP/CBC)
    participant TravelTime as service/travel_time.py
    participant SimEngine as simulation/engine.py
    participant ArrivalGen as demand/arrival_generator.py
    participant Dispatcher as simulation/dispatcher.py
    participant ServiceTime as service/service_time.py
    participant Metrics as simulation/metrics.py
    participant SciPy as scipy.stats
    participant Matplotlib as matplotlib

    User->>RunProdV2: python scripts/run_production_v2.py

    Note over RunProdV2: STEP 1   Generate Allocations
    RunProdV2->>Allocator: EMSAllocator.from_project(PROJECT_ROOT)
    Allocator->>TravelTime: build_travel_time_matrix()
    Note right of TravelTime: Loads distance_matrix_*.csv,<br/>firehouses_*.csv,<br/>configs/service.yaml
    Allocator-->>RunProdV2: allocator ready

    loop For each K in [10,15,20,25,30,35,40,45,48]
        RunProdV2->>Policies: spatially_stratified_allocation(K, cap=2)   P0
        RunProdV2->>Policies: demand_proportional_allocation(..., K, cap=2)   P1
        RunProdV2->>Allocator: allocator.solve("demand_weighted", K, cap=2)   P2
        Allocator->>Models: build_demand_weighted(travel_time, demand, K, cap=2)
        Models-->>Allocator: solved LpProblem
        RunProdV2->>RunProdV2: Save allocations_K{K}.csv
    end

    Note over RunProdV2: STEP 2   Run Simulations (810 runs)
    loop 3 policies   9 K values   30 replications
        RunProdV2->>SimEngine: EMSSimulation(allocation, config, seed)
        SimEngine->>ArrivalGen: NHPPArrivalGenerator(data_dir)
        Note right of ArrivalGen: Loads demand_lambda_*.csv
        SimEngine->>Dispatcher: NearestAvailableDispatcher(distance_matrix)
        SimEngine->>ServiceTime: ServiceTimeModel(mean=25, std=10)
        SimEngine->>SimEngine: sim.run(horizon_hours=168)
        Note right of SimEngine: SimPy event loop:<br/>arrival   dispatch   travel  
    service   complete
        SimEngine->>Metrics: collect response times, queues, utilization
        SimEngine-->>RunProdV2: results dict (summary + incident_log)
    end
    RunProdV2->>RunProdV2: Save all_results_raw.csv + per-K CSVs

    Note over RunProdV2: STEP 3   Statistical Analysis
    RunProdV2->>SciPy: f_oneway (ANOVA per K)
    RunProdV2->>SciPy: tukey_hsd (post-hoc pairwise)
    RunProdV2->>SciPy: Cohen's d effect sizes
    RunProdV2->>SciPy: t.interval (confidence intervals)
    RunProdV2->>RunProdV2: Save tables/*.csv

    Note over RunProdV2: STEP 4   Visualizations
    RunProdV2->>Matplotlib: Generate 10+ figure types
    RunProdV2->>RunProdV2: Save figures/*.png

    Note over RunProdV2: STEP 5   V1 Comparison
    RunProdV2->>RunProdV2: Load legacy results, compute deltas
    RunProdV2->>RunProdV2: Save comparison_with_v1.csv
```

Note over RunProdV2: STEP 6  Experiment Log
 RunProdV2->>RunProdV2: Append to experiment_log.txt
 RunProdV2-->>User: Pipeline complete. Results **in** results/baseline/

Alternate Path: Data Regeneration

```
sequenceDiagram
    participant User
    participant GenData as scripts/generate_all_data.py
    participant T1 as tier1_boundaries.py
    participant T2 as tier2_*.py
    participant T3 as tier3_*.py

    User->>GenData: python scripts/generate_all_data.py
    GenData->>GenData: verify_raw_data()
    GenData->>T1: process_boundaries(project_root)
    T1-->>GenData: geometry pickles created
    GenData->>T2: process_firehouses / process_precincts / process_crashes
    T2-->>GenData: Manhattan-filtered CSVs
    GenData->>T3: build_lambda_tables / build_distance_matrices
    T3-->>GenData: demand_lambda_*.csv, distance_matrix_*.csv
    GenData->>GenData: DataVersionManager.create_manifest()
    GenData-->>User: All processed data generated
```

Divergence Note

Data flow (View 2) shows artifact dependencies; runtime sequence (View 3) shows execution order. They align: data processing feeds optimization which feeds simulation. No divergence detected. **[Confirmed]**

G. Single Source of Truth Metadata Table

Component	File Path	Responsibility	Entry Point ?	Inputs	Outputs	Upstream Deps	Downstream Consumers	Execution Command	Status	Evidence
Production Pipeline	scripts/run_production_v2.py	Full production run: allocations + simulation + stats + viz	Yes	Processed data in data/processed/	results/baseline/**	src/ems_readiness/	Final deliverables	python scripts/run_production_v2.py	Active	Confirmed
Data Pipeline	scripts/generate_all_data.py	Generate all processed data from raw	Yes	data/raw/*	data/processed/*	Raw data files	Production pipeline	python scripts/generate_all_data.py	Active	Confirmed
Demand Modeling	scripts/demand_modeling.py	Standalone demand model fitting + visualization	Yes (standalone)	data/raw/Motor_Vehicle_Collisions_*.csv	Lambda tables, EDA figures	Raw crash data	—	python scripts/demand_modeling.py	Active	Confirmed
Verification	scripts/run_verification	V&V toy scenarios (4	Yes (standalone)	Processed data	results/simulation/verification/	Simulation engine	Verification log	python scripts/run_verification	Active	Confirmed

Component	File Path	Responsibility	Entry Point ?	Inputs	Outputs	Upstream Deps	Downstream Consumers	Execution Command	Status	Evidence
	catio n.py	cases)			fica- tion/ *.jso n			catio n.py		
Validation Pilots	scrip ts/ run_v alid- ation _pi- lots. py	Valid- ation pilot runs (3 pi- lots)	Yes (stan- dalon e)	Pro- cesse d data	res- ults/ basel ine/ simu- la- tion/ val- ida- tion_ pi- lot/*	Simu- lation en- gine, op- tim- izer	Valid- ation re- ports	py- thon scrip ts/ run_v alid- ation _pi- lots. py	Active	Con- firmed
Consistency Checker	scrip ts/ ver- ify_pr oject _con- sist- ency. py	End- to- end pro- ject con- sist- ency ver- ifica- tion	Yes (stan- dalon e)	All pro- ject files	Con- sole report	—	—	py- thon scrip ts/ ver- ify_pr oject _con- sist- ency. py	Utility	Con- firmed
Notebook Builder	scrip ts/ build _en- hance d_not ebook .py	Gen- erate en- hance d end- to- end note- book	Yes (stan- dalon e)	—	note- books / 01_en d_to_ end_w ork- flow. ipynb	—	Note- book users	py- thon scrip ts/ build _en- hance d_not ebook .py	Utility	Con- firmed

Component	File Path	Responsibility	Entry Point ?	Inputs	Outputs	Upstream Deps	Downstream Consumers	Execution Command	Status	Evidence
Notebook Fix	scripts/fix_notebook_nomenclature.py	Fix policy naming in notebooks	Yes (standardone)	Notebooks	Updated notebooks	—	—	python scripts/fix_notebook_nomenclature.py	Utility	Confirmed
Arrival Generator	src/ems_readiness/demand/arrival_generator.py	NHPP thinning algorithm for EMS call generation	No	demand_lambda*.csv	Arrival events (in-memory)	Lambda tables	Simulation engine	—	Active	Confirmed
Allocator	src/ems_readiness/optimization/allocator.py	High-level allocation interface; loads data + solves	No	Distance matrix, demand, configs	Allocation-Result	models.py, policies.py, travel_time.py	Production pipeline, notebooks	—	Active	Confirmed
Optimization	src/ems_readiness/	MIP formulations	No	Travel time matrix,	Solved PuLP	PuLP/CBC	allocator-	—	Active	Confirmed

Component	File Path	Responsibility	Entry Point ?	Inputs	Outputs	Upstream Deps	Downstream Consumers	Execution Command	Status	Evidence
Models	optimization/models.py	(demand_weighted, p_median, maximal_coverage)		demand	problem		ator.py			
Baseline Policies	src/ems_readiness/optimization/policies.py	P0 (spatial), P1 (demand-proportional), legacy uniform	No	Firehouse data, demand	Allocation Series	Processed data	Production pipeline	—	Active	Confirmed
Travel Time	src/ems_readiness/service/travel_time.py	Haversine distance / speed proxy for travel times	No	Distance matrix, configs/service.yaml	Travel time matrix	utils/distance.py	Allocator, dispatcher	—	Active	Confirmed
Service Time	src/ems_readiness/	LogNormal on-scene	No	configs/service	Service time	—	Simulation engine	—	Active	Confirmed

Component	File Path	Responsibility	Entry Point ?	Inputs	Outputs	Upstream Deps	Downstream Consumers	Execution Command	Status	Evidence
	service/service_time.py	service time sampling		vice.yaml	samples					
Simulation Engine	src/ems_readiness/simulation/engine.py	SimPy DES orchestration	No	Allocation, config, seed	Results dict (summary + incident log)	arrival_generator, dispatcher, service_time, metrics	Production pipeline	—	Active	Confirmed
Dispatcher	src/ems_readiness/simulation/dispatcher.py	Nearest-available dispatch logic	No	Distance matrix, unit pool	Dispatch decisions	resources.py, travel_time.py	Simulation engine	—	Active	Confirmed
Batch Runner	src/ems_readiness/simulation/run-	Multi-replication batch execution	No	Allocation, config	Aggregated results + CIs	Simulation engine	Validation pilots	—	Active	Confirmed

Component	File Path	Responsibility	Entry Point ?	Inputs	Outputs	Upstream Deps	Downstream Consumers	Execution Command	Status	Evidence
	ner.py									
Metrics Collector	src/ems_readiness/simulation/metrics.py	Response time, queue, utilization statistics	No	Incident events	Summary dict	entities.py	Simulation engine	—	Active	Confirmed
Resources	src/ems_readiness/simulation/resources.py	EMS unit pool and availability tracking	No	Allocation	Unit status	—	Dispatcher	—	Active	Confirmed
Entities	src/ems_readiness/simulation/entities.py	Incident data-class	No	—	—	—	Metrics, engine	—	Active	Confirmed
Distance Utils	src/ems_readiness/	Haversine and Man-	No	Coordinates	Distance (miles)	—	Travel time, data	—	Active	Confirmed

Component	File Path	Responsibility	Entry Point ?	Inputs	Outputs	Upstream Deps	Downstream Consumers	Execution Command	Status	Evidence
	utils/ distance.py	hatten distance functions					pipeline			
Reproducibility	src/ ems_readiness/ utils/ reproducibility.py	Seed management (Seed Manager)	No	configs/ reproducibility.yaml	Deterministic RNG	—	All stochastic components	—	Active	Confirmed
Data Processing (Tier 1)	scripts/ data_processing/ tier1_boundaries.py	Boundary geometry processing	No	GeoJSON boundary files	Geometry pickles	Raw GeoJSON	Tier 2 processors	—	Active	Confirmed
Data Processing (Tier 2)	scripts/ data_processing/ tier2_*.py	Spatial filtering to Manhattan subset	No	Raw CSVs + geometry pickles	Manhattan-filtered CSVs	Tier 1	Tier 3 modeling	—	Active	Confirmed
			No			Tier 2		—		

Component	File Path	Responsibility	Entry Point ?	Inputs	Outputs	Upstream Deps	Downstream Consumers	Execution Command	Status	Evidence
Data Processing (Tier 3)	scripts/data_processing/tier3/*.py	Demand lambda tables + distance matrices		Manhattan-filtered data	Lambda CSVs + distance CSVs		Production pipeline		Active	Confirmed
Cache Manager	scripts/data_processing/cache.py	Smart caching for data pipeline	No	File hashes	Cache decisions	—	generate_all_data.py	—	Utility	Confirmed
Validation	scripts/data_processing/validation.py	Data validation checks	No	Processed data	Validation errors	—	generate_all_data.py	—	Utility	Confirmed
Versioning	scripts/data_processing/versioning.py	Data manifest generation	No	Processed data + git	Manifest JSON	—	generate_all_data.py	—	Utility	Confirmed

Component	File Path	Responsibility	Entry Point ?	Inputs	Outputs	Upstream Deps	Downstream Consumers	Execution Command	Status	Evidence
End-to-End Notebook	notebooks / 01_end_to_end_workflow.ipynb	Complete pipeline in notebook form	Yes (notebook)	Raw data (or processed)	All outputs	src/ems_readiness/	Interactive exploration	Jupyter	Active	Confirmed
EDA Notebook	notebooks / 02_eda_spatiotemporal.ipynb	Spatiotemporal exploratory analysis	Yes (notebook)	Processed data	Visualizations	Processed data	—	Jupyter	Active	Inferred
Input Modeling Notebook	notebooks / 03_input_modeling.ipynb	Demand model fitting exploration	Yes (notebook)	Crash data	Model fits	Processed data	—	Jupyter	Active	Inferred
Optimization Notebook	notebooks / 05_optimization.ipynb	Interactive optimization exploration	Yes (notebook)	Processed data	Allocations	src/ems_readiness/optimization/	—	Jupyter	Active	Inferred
Produc-	notebooks	Results		results/		Produc-	—			

Component	File Path	Responsibility	Entry Point ?	Inputs	Outputs	Upstream Deps	Downstream Consumers	Execution Command	Status	Evidence
tion Results Notebook	/07_production_results.ipynb	visualization and analysis	Yes (notebook)	baseline/	Visualizations	tion outputs		Jupyter	Active	Inferred
Statistical Analysis Notebook	notebooks / 08_statistical_analysis.ipynb	ANOVA, effect sizes, post-hoc tests	Yes (notebook)	results/baseline/	Statistical tables	Production outputs	—	Jupyter	Active	Inferred
CBD Analysis Notebook	notebooks / 09_cbd_analysis.ipynb	CBD-focused analysis	Yes (notebook)	CBD experiment results	CBD comparison	Analysis outputs	—	Jupyter	Active	Inferred
Colab Pipeline	notebooks / colab_standalone/ EMS_Optimization_Com-	Self-contained Colab version	Yes (notebook)	Embedded/downloaded data	All outputs	—	Colab users	Google Colab	Active	Confirmed

Component	File Path	Responsibility	Entry Point ?	Inputs	Outputs	Upstream Deps	Downstream Consumers	Execution Command	Status	Evidence
	plete_Pipeline.ipynb									
Analysis Scripts	scripts/analysis/*.py (19 files)	Supporting robustness & sensitivity analyses	Yes (standalone each)	Processed data + baseline results	results/analysis/**	Production outputs	Analysis docs	python scripts/analysis/<name>.py	Active	Confirmed
Legacy Production	scripts/archive/run_production_experiments.py	Original (V1) production experiments	No (archived)	Processed data	Legacy results	—	V1 comparison in run_production_v2.py	—	Legacy	Confirmed
Legacy Audit Scripts	scripts/archive/audit_utilities/*.py , data_audit.py	Historical data audit utilities	No (archived)	Raw/processed data	Audit reports	—	—	—	Legacy	Confirmed



H. Pipeline Inventory and Primary Path Determination

All Identified Pipelines / Entry Points

#	Path	Type	Description	Status	Primary?
1	python scripts/ run_production_v2.py	Script	Full production pipeline: 3 policies × 9 K values × 30 reps + stats + viz	Active	✅ YES — Golden Path
2	python scripts/gen- erate_all_da ta.py	Script	Data pro- cessing pipeline (raw → processed)	Active	Prerequisite to #1
3	python scripts/ run_verifica tion.py	Script	4 V&V toy scenarios	Active	Supporting
4	python scripts/ run_validati on_pilots.py	Script	3 validation pilots (P0 vs P2, K sensit- ivity, demand sensitivity)	Active	Supporting
5	notebooks/ 01_end_to_en d_workflow.i pynb	Notebook	Complete pipeline in in- teractive form	Active	Notebook convenience path
6	notebooks/ 02_eda_spati otempor- al.ipynb	Notebook	Exploratory data analysis	Active	Analysis
7	notebooks/ 03_input_mod eling.ipynb	Notebook	Demand model fitting	Active	Analysis
8	notebooks/ 04_service_t ravel_proxy. ipynb	Notebook	Service/ travel time modeling	Active	Analysis
9	notebooks/ 05_optimizat ion.ipynb	Notebook	Optimization exploration	Active	Analysis

#	Path	Type	Description	Status	Primary?
10	notebooks/ 06_simulation_debug.ipynb	Notebook	Simulation debugging	Active	Debug
11	notebooks/ 07_production_results.ipynb	Notebook	Production results visualization	Active	Analysis
12	notebooks/ 08_statistical_analysis.ipynb	Notebook	Statistical testing	Active	Analysis
13	notebooks/ 09_cbd_analysis.ipynb	Notebook	CBD-focused analysis	Active	Analysis
14	notebooks/ colab_standalone/ EMS_Optimization_Complete_Pipeline.ipynb	Notebook	Google Colab self-contained pipeline	Active	Colab convenience
15	scripts/analysis/capacity_sensitivity_analysis.py	Script	Capacity sensitivity sweep (cap 1-5)	Active	Robustness analysis
16	scripts/analysis/run_cbd_experiment.py	Script	CBD robustness experiment	Active	Robustness analysis
17	scripts/analysis/run_distance_comparison_experiment.py	Script	Haversine vs Manhattan distance comparison	Active	Alternative analysis

#	Path	Type	Description	Status	Primary?
18	scripts/analysis/run_cbd_focused_optimization.py	Script	CBD-focused optimization comparison	Active	Alternative analysis
19	scripts/analysis/generate_publication_figures.py	Script	Generate pub-quality figures	Active	Visualization
20	scripts/analysis/regenerate_all_figures.py	Script	Batch figure regeneration	Active	Visualization
21	scripts/archive/run_production_experiments.py	Script	Original V1 production pipeline	Legacy	No — superseded by #1
22	scripts/archive/audit_step*.py	Script	Historical data auditing (4 scripts)	Legacy	No
23	scripts/archive/data_audit.py	Script	Historical data audit	Legacy	No
24	pytest tests/ -v	Test suite	13 test modules	Active	Quality assurance

Primary Path Determination

Golden Path: `scripts/run_production_v2.py`

- **Why:** This is the single script that produces all canonical baseline results (`results/baseline/`), which are the source of truth cited in the technical report and README. It orchestrates allocation generation, full simulation suite, statistical analysis, and visualization in one run.

- **Evidence:** `README.md` “Start Here” section directs users to this script. `results/WHICH_FILES_TO_USE.md` identifies `results/baseline/` as the source of truth.

Notebook Convenience Path: `notebooks/01_end_to_end_workflow.ipynb`

- **Why:** Provides the same pipeline in interactive notebook form for exploration and teaching.

Source of Truth for Reproducibility: `scripts/run_production_v2.py` (script) over notebooks, because the script uses hardcoded, auditable constants and produces deterministic file outputs.

I. Git LFS / Data Availability Dependencies

LFS-Tracked Files

File	Size (approx)	Required For	Evidence
<code>data/raw/Motor_Vehicle_Collisions_-_Crashes_20260223.csv</code>	~2.24M rows, ~400MB	<code>generate_all_data.py</code> (data regeneration only)	<code>.gitattributes</code>
<code>data/raw/Police_Precincts_20260223.csv</code>	Small-medium	<code>generate_all_data.py</code> (data regeneration only)	<code>.gitattributes</code>

Minimum Required Data for the Golden Path

The Golden Path (`run_production_v2.py`) requires these processed data files, **all of which are committed to the repository** as `.gitignore` exceptions:

File	Role	Committed?
<code>data/processed/demand_lambda_hourly.csv</code>	NHPP hourly intensity factors	✓ Yes
<code>data/processed/demand_lambda_dow.csv</code>	Day-of-week factors	✓ Yes
<code>data/processed/demand_lambda_precinct.csv</code>	Per-precinct base rates	✓ Yes
<code>data/processed/distance_matrix_firehouse_precinct.csv</code>	Firehouse-to-precinct distances	✓ Yes
<code>data/processed/firehouses_manhattan.csv</code>	Manhattan firehouse coordinates	✓ Yes
<code>data/processed/firehouses_clean.csv</code>	Cleaned firehouse data	✓ Yes
<code>data/processed/precincts_manhattan.geojson</code>	Precinct boundaries	✓ Yes
<code>data/processed/demand_model_summary.json</code>	Demand model parameters	✓ Yes

Consequences of Missing Data

Scenario	Impact
LFS files not pulled (crash CSV, precincts CSV)	<code>generate_all_data.py</code> will fail at Tier 2. Golden Path unaffected (processed data already committed).
Processed data deleted	<code>run_production_v2.py</code> will fail. Fix: <code>python scripts/generate_all_data.py</code> (requires LFS files).
Both LFS and processed data missing	Complete pipeline blocked. Must pull LFS files first, then regenerate data.
<code>configs/</code> files missing	Scripts may fail or use hardcoded defaults. <code>run_production_v2.py</code> hardcodes most constants, so partial resilience exists for that script. [Confirmed — constants in script override configs]

J. Minimal Reproducible Path

Goal: Produce one valid output (allocation + simulation results for K=20) with the smallest footprint.

Minimal File Set

```
ems-optimization/
├── configs/
│   ├── demand.yaml
│   ├── optimization.yaml
│   ├── service.yaml
│   └── simulation.yaml
├── data/processed/
│   ├── demand_lambda_hourly.csv
│   ├── demand_lambda_dow.csv
│   ├── demand_lambda_precinct.csv
│   ├── distance_matrix_firehouse_precinct.csv
│   ├── firehouses_manhattan.csv
│   └── firehouses_clean.csv
├── src/ems_readiness/ (entire package)
├── scripts/run_production_v2.py
└── requirements.txt
```

Minimal Command

```
pip install -r requirements.txt
python scripts/run_production_v2.py --reps 1
```

Note: `--reps 1` reduces from 30 to 1 replication per scenario, cutting runtime from ~30+ minutes to ~3 minutes. The script still runs all 9 K values and 3 policies (27 scenarios × 1 rep = 27 runs). **[Confirmed — `argparse` definition in `run_production_v2.py`]**

Observable Success Indicators

1. `results/baseline/allocations/allocations_K20.csv` exists with 3 columns (P0, P1, P2) and 48 rows
2. `results/baseline/simulation/all_results_raw.csv` exists with performance metrics
3. `results/baseline/tables/descriptive_statistics.csv` exists
4. `results/baseline/figures/mean_rt_vs_K.png` exists
5. `results/baseline/experiment_log.txt` contains run summary
6. Console output shows no `ERROR` lines

Shortcuts / Defaults

- `--skip-sim` flag on `run_production_v2.py` loads existing simulation results and only reruns stats + viz. **[Confirmed — `argparse`]**
- Processed data is pre-committed, so `generate_all_data.py` can be skipped on fresh clone (if LFS is not needed).

K. Ambiguities, Gaps, and Risk Register

#	Category	Description	Severity	Evidence	Mitigation
1	Config vs Code Override	<code>run_production_v2.py</code> hardcodes <code>CAPACITY=2</code> , <code>K_VALUES</code> , <code>HORIZON_HOURS</code> , etc., potentially diverging from <code>configs/*.yaml</code> values. The script's hard-coded values take precedence.	Low	Confirmed — code inspection	Document that script constants are authoritative for production runs; configs are used by library code and notebooks.
2	Missing <code>src/ems_readiness/analysis.py</code>	Makefile target <code>analysis</code> runs <code>python src/ems_readiness/analysis.py</code> , but this file does not exist in the repository.	Medium	Confirmed — file listing shows no such file	The <code>make analysis</code> target is non-functional. Use <code>run_production_v2.py</code> instead.
3	Notebook Execution State Unknown	Notebooks (02 through 09) are present but their execution state (whether cells are up-to-date with current codebase) is not verified. <code>.py</code> mirror files exist for some (05 ,	Low	Inferred — <code>.py</code> files and <code>.ipynb</code> coexist	Run notebooks fresh with <code>Restart & Run All</code> to verify.

#	Category	Description	Severity	Evidence	Mitigation
		<code>06</code> , <code>07</code> , <code>08</code>).			
4	Legacy <code>uniform_allocation</code> Still Importable	<code>policies.py</code> retains deprecated <code>uniform_allocation()</code> marked with warnings. Could confuse new users.	Low	Confirmed — source code	Deprecation warnings are present. Consider removing in future release.
5	<code>crashes_manifest.csv</code> Not Committed	The intermediate crash data (~109MB) is gitignored. If a user needs to analyze crash-level data without raw LFS files, they cannot.	Low	Confirmed — <code>.gitignore</code>	Regenerate via <code>make data</code> if LFS files available.
6	Colab Notebooks Data Dependency	<code>notebooks/colab_standalone/</code> notebooks may embed data download logic whose URLs could become stale.	Low	Inferred — typical Colab pattern	Verify Colab notebooks periodically.
7	No <code>setup.py</code> / <code>pyproject.toml</code>	The <code>src/ems_readiness/</code> package has no packaging configuration. It relies on	Low	Confirmed — no setup files found	Consider adding <code>pyproject.toml</code> with <code>pip install -e .</code>

#	Category	Description	Severity	Evidence	Mitigation
		<code>sys.path.insert()</code> hacks in every script.			for cleaner imports.
8	data/processed/.cache_manifest.json and .data_manifest.json	These are generated by the data pipeline and gitignored, but are present in the repo (perhaps committed before gitignore rule). May cause confusion about whether they're authoritative.	Low	Confirmed — files exist but pattern is git-ignored	Clean up or explicitly commit/ignore.
9	Test Data Dependency	Tests in <code>tests/</code> depend on <code>data/processed/</code> files being present. If processed data is missing, tests fail.	Medium	Confirmed — <code>conftest.py</code> loads from <code>data/processed/</code>	Ensure <code>make data</code> or committed processed files are available before running tests.
10	results/simulation/ vs results/baseline/simulation/	Both directories exist. <code>results/simulation/</code> contains only verification JSONs. <code>results/baseline/simulation/</code> contains	Low	Confirmed — file listing	<code>WHICH_FILES_TO_USE.md</code> clarifies this.

#	Category	Description	Severity	Evidence	Mitigation
		production results. Potential path confusion.			
11	<code>__pycache__</code> in <code>scripts/</code> and <code>scripts/analysis/</code>	Compiled Python caches exist for scripts both in root <code>scripts/</code> <code>__pycache__/</code> and <code>scripts/analysis/</code> <code>__pycache__/</code> , suggesting scripts were previously run from root <code>scripts/</code> before the analysis/ reorganization.	Low	Confirmed — file listing	Add <code>__pycache__/</code> cleanup to <code>make clean</code> .
12	Distance Matrix Variant	Two distance matrices exist: <code>distance_matrix_firehouse_precinct.csv</code> and <code>distance_matrix_firehouse_precinct_manhattan.csv</code> . The relationship and which is canonical is not immediately clear from file-names alone.	Low	Inferred — both files committed	Likely the <code>_manhattan</code> variant is Manhattan-distance metric vs Haversine. Used by distance comparison analysis.
13			Low		

#	Category	Description	Severity	Evidence	Mitigation
	Simulation horizon = 1 week	The 168-hour horizon is a modeling choice. Whether this is sufficient for steady-state behavior depends on demand rates. Warmup is set to 0 hours (terminating simulation).		Confirmed — <code>configs/simulation.yaml</code> , <code>run_production_v2.py</code>	Documented in experimental design. 30 replications compensate for single-week horizon.

Assumptions Directly Testable from Repository Evidence

Assumption	Testable?	How
P2 outperforms P0 at K=20	✓ Yes	Compare mean_RT in <code>results/baseline/tables/descriptive_statistics.csv</code>
Capacity=2 is optimal	✓ Yes	Review <code>results/analysis/capacity_comparison/full_comparison.csv</code>
All 810 production runs completed	✓ Yes	Check <code>wc -l results/baseline/simulation/all_results_raw.csv</code> (expect 811 = header + 810 rows)
ANOVA shows significant differences	✓ Yes	Check <code>p_value < 0.05</code> in <code>results/baseline/tables/anova_results.csv</code>
Seeds are deterministic	✓ Yes	Re-run with <code>--reps 1</code> and compare output to existing results
Haversine vs Manhattan distance gives similar results	✓ Yes	Review <code>results/analysis/distance_comparison/comparison_table.csv</code>

What Remains Unknown

Item	Status
Real-world validation against actual NYC EMS response times	Unknown — no external validation data in repo
Deployment / operational integration plan	Unknown — repo is research/analysis only
CI/CD pipeline	Unknown — no <code>.github/workflows/</code> , no CI config found
Performance benchmarks (wall-clock time for full pipeline)	Unknown — not systematically recorded
Whether <code>make analysis</code> target was ever functional	Unknown — <code>src/ems_readiness/analysis.py</code> does not exist

End of Architectural Map